BÁO CÁO ĐỒ ÁN

AN TOÀN ỨNG DỤNG WEB

ĐỀ TÀI B22

NGHIÊN CỨU VÀ THỰC NGHIỆM LỖ HỔNG TẢI TỆP KHÔNG AN TOÀN TRÊN PHP/APACHE VÀ NODE.JS/EXPRESS

PHÁT HIỆN VÀ PHÒNG VỆ NHIỀU LỚP

Báo cáo kỹ thuật dựa trên hệ thống B22 UFU Lab

Mã học phần: NT213

Năm 2026

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU....4
1.1. Bối cảnh và lý do chọn đề tài....4
1.2. Mục tiêu và đóng góp....4
1.3. Câu hỏi nghiên cứu và phạm vi....4
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT....5
2.1. Quy trình tải tệp và ranh giới tin cậy....5
2.2. Các cơ chế kiểm tra yếu....5
2.3. Traversal, polyglot và xử lý nhiều bước....6
2.4. Nguyên tắc phòng vệ nhiều lớp....6
2.5. Phân tích tĩnh và kiểm thử động....7
CHƯƠNG 3. KHẢO SÁT CÔNG TRÌNH LIÊN QUAN....7
3.1. Tiêu chí lựa chọn tài liệu....7
3.2. UChecker — phát hiện upload không hạn chế trên PHP....8
3.3. Nghiên cứu thực nghiệm của Riadi và Aristianto....8
3.4. Pixy — phân tích tĩnh ứng dụng web....8
3.5. Saner — kết hợp phân tích tĩnh và động....9
3.6. Phát hiện lỗ hổng bậc hai....9
3.7. Tổng hợp và vị trí của đề tài....10
CHƯƠNG 4. THỰC NGHIỆM....11
4.1. Thiết kế nghiên cứu và môi trường....11
4.2. Mô hình đe dọa....11
4.3. Kiến trúc hệ thống....12
4.4. Các kịch bản có lỗ hổng....13
4.5. Luồng phòng vệ và triển khai....14
4.6. Quy trình tái lập và tiêu chí đánh giá....15
4.7. Kịch bản trình diễn....15

CHƯƠNG 5. KẾT QUẢ VÀ BÀN LUẬN 16
5.1. Nguồn bằng chứng và trạng thái xác minh 16
5.2. Kết quả của nhánh PHP 16
5.3. Tổng hợp kết quả đã lưu 17
5.4. Kết quả Node.js và Semgrep 17
5.5. Trả lời câu hỏi nghiên cứu và hạn chế 18
5.6. Các nguy cơ ảnh hưởng tính đúng đắn 18
5.7. Ý nghĩa thực tiễn 19
CHƯƠNG 6. KẾT LUẬN 19
6.1. Tổng kết 19
6.2. Hướng phát triển 20
TÀI LIỆU THAM KHẢO 21

Phạm vi tài liệu: báo cáo sử dụng mã nguồn, cấu hình và bằng chứng thực nghiệm có trong B22 UFU Lab. Kết quả tích hợp đã lưu được phân biệt với kiểm tra chạy lại khi biên soạn; không xem kiểm thử chưa thực hiện là kết quả đạt.

CHƯƠNG 1. GIỚI THIỆU

1.1. Bối cảnh và lý do chọn đề tài

Tải tệp là chức năng phổ biến của ứng dụng web, phục vụ ảnh đại diện, tài liệu và dữ liệu do người dùng cung cấp. Khác với một trường văn bản đơn giản, một yêu cầu upload chứa nhiều yếu tố cần xử lý đồng thời: tên tệp, phần mở rộng, Content-Type, kích thước, nội dung nhị phân và vị trí lưu trữ. Các thuộc tính này không nhất thiết nhất quán và phần lớn nằm trong sự kiểm soát của phía gửi. Vì vậy, việc ứng dụng chấp nhận một tệp không đồng nghĩa tệp đó an toàn để lưu, phục vụ hoặc xử lý.

Lỗ hổng tải tệp không an toàn xuất hiện khi dữ liệu không tin cậy vượt qua chính sách kiểm tra và được sử dụng trong ngữ cảnh nguy hiểm. CWE-434 mô tả trường hợp cho phép tải lên tệp thuộc loại nguy hiểm có thể được xử lý tự động trong môi trường sản phẩm [6]. Hậu quả phụ thuộc vào cách máy chủ sử dụng tệp: phục vụ nội dung không mong muốn, ghi sai vị trí, tiêu hao tài nguyên hoặc diễn giải mã phía máy chủ. Do đó, nghiên cứu cần bao quát cả khâu tiếp nhận lẫn các bước sau khi lưu.

Đề tài lựa chọn PHP/Apache và Node.js/Express vì hai nền tảng có cách xử lý tệp khác nhau. Apache trong lab được cấu hình để minh họa việc diễn giải PHP trong thư mục upload, trong khi Express phục vụ các tệp thuộc thư mục static. Việc đối chiếu giúp tránh suy luận rằng một kỹ thuật upload luôn tạo ra cùng tác động trên mọi nền tảng.

1.2. Mục tiêu và đóng góp

Mục tiêu của đồ án là xây dựng môi trường thực nghiệm có thể tái lập, phân tích các cơ chế kiểm tra yếu, triển khai nhánh phòng vệ nhiều lớp và đối chiếu bằng chứng thu được. Đóng góp ở quy mô môn học gồm hai ứng dụng lab độc lập, bộ kiểm thử HTTP tự động, hai quy tắc Semgrep và quy trình tổng hợp kết quả. Đề tài không đề xuất thuật toán phát hiện mới và không tuyên bố giải quyết toàn bộ lớp lỗ hổng upload.

1.3. Câu hỏi nghiên cứu và phạm vi

Bốn câu hỏi được đặt ra: (RQ1) các kiểm tra đơn lẻ thất bại như thế nào trong bộ kịch bản đã thiết kế; (RQ2) cấu hình máy chủ ảnh hưởng thế nào đến tác động sau upload; (RQ3) kiểm tra nội dung, tái mã hóa và tách biệt lưu trữ có đạt hành vi mong

đợi trong lab không; (RQ4) các quy tắc Semgrep phát hiện được mẫu nào và còn bỏ ngỏ điều gì.

Thực nghiệm chỉ diễn ra trên máy cục bộ. Payload PHP chỉ in chuỗi đánh dấu, không chạy lệnh hệ điều hành; mẫu HTML chỉ chứa marker. Không sử dụng dữ liệu thật và không triển khai các endpoint cố ý có lỗ hổng lên Internet. Báo cáo phân biệt kết quả quan sát với dự đoán và phân biệt bản mã hiện tại với bằng chứng của lần chạy trước.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Quy trình tải tệp và ranh giới tin cậy

Một yêu cầu multipart/form-data chia nội dung HTTP thành các phần theo boundary. Phần chứa tệp thường có tên field, filename và Content-Type. Ở PHP, thông tin upload được cung cấp qua cấu trúc $_FILES và tệp tạm; ở Node.js, middleware Multer phân tích multipart và cung cấp đối tượng req.file. Các framework hỗ trợ tiếp nhận dữ liệu nhưng không tự quyết định toàn bộ chính sách bảo mật phù hợp nghiệp vụ.

Ranh giới tin cậy đầu tiên nằm giữa client và ứng dụng. Tên tệp và MIME do client khai báo chỉ là dữ liệu đầu vào, không phải bằng chứng định dạng. Ranh giới thứ hai nằm giữa ứng dụng và hệ thống tệp: một tên không được chuẩn hóa có thể làm thay đổi vị trí ghi. Ranh giới thứ ba nằm ở cách máy chủ hoặc ứng dụng đọc tệp đã lưu. Một ảnh được phục vụ như dữ liệu có ngữ cảnh khác với cùng tệp được đưa vào include của PHP.

2.2. Các cơ chế kiểm tra yếu

Kiểm tra Content-Type do người dùng gửi có thể bị đánh lừa vì client được quyền lựa chọn giá trị này. Kiểm tra phần mở rộng cũng chỉ xác nhận tên, không xác nhận nội dung. Khi có nhiều phần mở rộng, hành vi diễn giải của máy chủ còn phụ thuộc cấu hình handler. Bộ lọc xóa một chuỗi cấm khỏi tên tệp có thể tạo ra tên mới không như dự kiến, đồng thời không bảo đảm đã loại bỏ mọi cách biểu diễn nguy hiểm.

Magic bytes giúp nhận diện dấu hiệu đầu tệp nhưng không chứng minh toàn bộ tệp hợp lệ. Một tệp bắt đầu bằng chữ ký ảnh có thể hỏng cấu trúc hoặc chứa dữ liệu bổ sung. Việc đọc kích thước ảnh là một bước kiểm tra hữu ích nhưng không thay thế giải mã đầy đủ. Trong lab, ảnh có dữ liệu PHP nối thêm được dùng để minh họa sự khác biệt giữa việc nhận diện là ảnh và việc an toàn trong mọi ngữ cảnh xử lý.

2.3. Traversal, polyglot và xử lý nhiều bước

Path traversal phát sinh khi thành phần đường dẫn do người dùng cung cấp được kết hợp vào đường dẫn đích mà không giới hạn vị trí thực tế sau chuẩn hóa. Tác động cần được xác minh bằng vị trí ghi chứ không chỉ bằng mã HTTP. Polyglot hoặc nội dung phụ trong ảnh cần được diễn giải thận trọng: dữ liệu có thể không ảnh hưởng bộ đọc ảnh nhưng vẫn được một bộ diễn giải khác sử dụng. Kịch bản upload kết hợp LFI trong đồ án minh họa chuỗi xử lý nhiều bước này.

Null-byte trong đồ án là mô phỏng việc kiểm tra một tên rồi sử dụng tên bị cắt ở vị trí khác. Nó có giá trị giải thích lỗi thiết kế kiểm tra–sử dụng không nhất quán, nhưng không phải bằng chứng rằng PHP 8.2 chấp nhận null byte trong đường dẫn.

2.4. Nguyên tắc phòng vệ nhiều lớp

OWASP khuyến nghị phối hợp nhiều cơ chế thay vì dựa vào một bộ lọc duy nhất [7]. Chính sách cần xuất phát từ nghiệp vụ: trong đồ án, nhánh phòng vệ chỉ chấp nhận JPEG và PNG. Danh sách cho phép nhỏ giúp thu hẹp phạm vi xử lý, nhưng phải kết hợp đối chiếu nội dung và sử dụng thư viện giải mã tương ứng. Phần mở rộng hợp lệ chỉ là một điều kiện cần.

Sau khi giải mã ảnh, hệ thống tái mã hóa thành ảnh đầu ra mới. Cách này tạo lại biểu diễn ảnh từ dữ liệu mà bộ giải mã hiểu, thay vì giữ nguyên toàn bộ byte đầu vào. Dữ liệu nối thêm không thuộc ảnh có thể bị loại bỏ trong quá trình này. Tuy nhiên, đây không phải bộ quét malware tổng quát; thư viện xử lý ảnh vẫn cần cập nhật, và khả năng loại bỏ một marker không chứng minh an toàn đối với mọi mẫu đầu vào.

Tên lưu nên do máy chủ sinh ngẫu nhiên, không dùng trực tiếp filename của client. Vùng lưu nên tách khỏi webroot hoặc thư mục static để không bị truy cập trực tiếp qua ánh xạ URL. Nếu nghiệp vụ yêu cầu tải xuống, ứng dụng cần một endpoint kiểm soát truy xuất. Header Content-Disposition dạng attachment và X-Content-Type-

Options: nosniff hỗ trợ hạn chế cách trình duyệt xử lý dữ liệu, nhưng không thay thế kiểm soát quyền đọc.

Bảng 2.1. Vai trò và giới hạn của các lớp phòng vệ

Lớp kiểm soát	Mục đích	Giới hạn
Extension và nhận diện nội dung	Giảm loại tệp ngoài chính sách	Không đủ nếu chỉ kiểm tra dấu hiệu đầu tệp
Giải mã và tái mã hóa	Tạo lại ảnh đầu ra hợp lệ	Phụ thuộc thư viện và tài nguyên xử lý
Giới hạn byte và pixel	Giảm chi phí bộ nhớ và CPU	Không thay thế rate limit hoặc quota
Tên ngẫu nhiên, lưu tách biệt	Giảm thao túng đường dẫn và truy cập trực tiếp	Không tự cung cấp phân quyền
Header tải xuống	Hạn chế diễn giải nội dung ở trình duyệt	Cần kết hợp xác thực và chính sách truy cập

2.5. Phân tích tĩnh và kiểm thử động

Phân tích tĩnh kiểm tra mã nguồn mà không cần thực thi từng luồng HTTP. Semgrep có thể phát hiện các mẫu cấu trúc được mô tả bằng rule. Trong lab, rule nhận diện lệnh chuyển tệp vào đường dẫn upload và cấu hình Multer lưu vào thư mục public. Các rule này không thực hiện chứng minh đầy đủ về luồng dữ liệu, điều kiện kiểm tra hoặc cấu hình triển khai. Kiểm thử động bổ sung bằng chứng hành vi thực tế; kết hợp hai cách tiếp cận giúp giảm kết luận chỉ dựa trên hình thức mã nguồn.

CHƯƠNG 3. KHẢO SÁT CÔNG TRÌNH LIÊN QUAN

3.1. Tiêu chí lựa chọn tài liệu

Năm công trình được lựa chọn bao quát hai hướng: nghiên cứu trực tiếp về tải tệp không an toàn và phương pháp phân tích bảo mật ứng dụng web có liên quan. Thông tin thư mục được đối chiếu qua DOI/Crossref hoặc trang hội nghị USENIX. Phần khảo sát tập trung vào vấn đề, hướng tiếp cận và ý nghĩa đối với đồ án; không gán số liệu thực nghiệm cho những nguồn chưa có toàn văn được đối chiếu. OWASP và CWE được sử dụng làm tài liệu kỹ thuật bổ trợ, không tính vào năm công trình học thuật.

3.2. UChecker — phát hiện upload không hạn chế trên PHP

Huang, Li, Zhang và Dai công bố UChecker tại DSN 2019 với tiêu đề "UChecker: Automatically Detecting PHP-Based Unrestricted File Upload Vulnerabilities" [1]. Công trình đặt bài toán tự động phát hiện UFU trong ứng dụng PHP, trùng với một phần đối tượng của đồ án. Đây là nguồn trực tiếp giúp xác định rằng việc nhận diện upload không an toàn cần được xem như một bài toán phân tích chức năng, thay vì chỉ liệt kê phần mở rộng nguy hiểm.

Đối với B22 UFU Lab, liên hệ chính là yêu cầu kiểm chứng cơ chế tiếp nhận và xử lý tệp trong ứng dụng PHP. Tuy nhiên, hai rule cấu trúc của lab không được coi là hiện thực lại UChecker, và báo cáo không đưa ra so sánh định lượng với công cụ này. Trong phạm vi nguồn đã xác minh, thông tin chắc chắn gồm tên tác giả, tiêu đề, hội nghị và DOI; chi tiết thuật toán hoặc số lượng lỗi của UChecker không được sử dụng làm kết luận của báo cáo.

3.3. Nghiên cứu thực nghiệm của Riadi và Aristianto

Riadi và Aristianto nghiên cứu "An Analysis of Vulnerability Web Against Attack Unrestricted Image File Upload", công bố năm 2016 [2]. Tóm tắt mô tả quá trình thu thập dữ liệu, phân tích trạng thái hiện tại, thiết kế cải tiến mã nguồn, kiểm thử và triển khai bản vá. Cách tổ chức trước–sau bản vá phù hợp với việc xây dựng hai nhánh có lỗ hổng và có phòng vệ trong đồ án.

Một điểm cần lưu ý là các tỷ lệ đánh giá trong tóm tắt công trình gắn với phản hồi của người tham gia, không tương đương tỷ lệ chặn payload trong bộ kiểm thử tự động. Đồ án vì vậy sử dụng hành vi quan sát được như vị trí lưu, mã HTTP, marker và header để kết luận từng test. Giá trị kế thừa là thiết kế đánh giá đối chứng; kết quả của nghiên cứu không được chuyển thành số liệu của lab.

3.4. Pixy — phân tích tĩnh ứng dụng web

Jovanovic, Kruegel và Kirda giới thiệu Pixy tại IEEE Symposium on Security and Privacy năm 2006 [3]. Công trình thuộc hướng sử dụng phân tích tĩnh để phát hiện lỗ hổng trong ứng dụng web. Hướng tiếp cận này cho thấy giá trị của việc kiểm tra đường đi của dữ liệu không tin cậy tới các thao tác nhạy cảm, thay vì chỉ dựa vào một yêu cầu kiểm thử cụ thể.

Liên hệ với đồ án là cách phân biệt dấu hiệu đáng ngờ và bằng chứng khai thác. Rule Semgrep có thể cảnh báo tại thao tác lưu tệp, nhưng muốn đánh giá chính sách upload cần xem thêm bước kiểm tra trước đó và ngữ cảnh xử lý sau đó. Pixy được sử dụng làm nền tảng phương pháp luận cho phần SAST, không được mô tả là công cụ được cài đặt hoặc chạy trong B22 UFU Lab. Khả năng của một bộ phân tích luồng dữ liệu cũng không được gán cho hai rule khớp mẫu hiện tại.

3.5. Saner — kết hợp phân tích tĩnh và động

Balzarotti và cộng sự trình bày "Saner: Composing Static and Dynamic Analysis to Validate Sanitization in Web Applications" tại IEEE Symposium on Security and Privacy năm 2008 [4]. Công trình tập trung vào kiểm chứng thao tác làm sạch dữ liệu bằng cách kết hợp phân tích tĩnh và động. Ý tưởng phương pháp phù hợp với tình huống một hàm lọc xuất hiện trong mã nhưng hiệu lực thực tế còn phụ thuộc đầu vào và ngữ cảnh sử dụng.

Trong lab, bộ lọc xóa chuỗi trong filename là ví dụ điển hình: sự có mặt của thao tác xử lý chuỗi không đủ chứng minh tên đầu ra an toàn. Kiểm thử HTTP và quan sát tệp giúp kiểm chứng kết quả biến đổi. Đồ án kế thừa nguyên tắc kết hợp hai nguồn bằng chứng, nhưng không hiện thực thuật toán Saner và không sử dụng bộ dữ liệu hoặc kết quả của công trình này như kết quả của nhóm.

3.6. Phát hiện lỗ hổng bậc hai

Dahse và Holz công bố "Static Detection of Second-Order Vulnerabilities in Web Applications" tại USENIX Security 2014 [5]. Công trình phân tích trường hợp payload được lưu trước, sau đó được sử dụng trong thao tác nhạy cảm. Tác giả liên kết các điểm đọc và ghi trong kho dữ liệu bền vững để tìm luồng dữ liệu chưa được làm sạch; trang hội nghị báo cáo 159 lỗ hổng bậc hai trong sáu ứng dụng web.

Mối liên hệ với đề tài là chuỗi upload–lưu trữ–xử lý lại: nguy hiểm không nhất thiết xảy ra tại request upload. Kịch bản LFI giới hạn trong thư mục uploads minh họa rằng tệp đã lưu vẫn là dữ liệu không tin cậy khi được đưa vào include. Tuy nhiên, đây chỉ là liên hệ về nguyên lý xử lý nhiều bước; lab không triển khai bộ phân tích kho dữ liệu như nghiên cứu của Dahse và Holz.

3.7. Tổng hợp và vị trí của đề tài

Bảng 3.1. So sánh năm công trình liên quan

Nguồn	Trọng tâm	Liên hệ với B22	Giới hạn khi đối chiếu
UChecker [1]	Tự động phát hiện UFU trên PHP	Bài toán phát hiện trực tiếp	Không tái hiện công cụ; không so sánh số lỗi
Riadi và Aristianto [2]	Upload ảnh và đánh giá trước–sau bản vá	Hai nhánh đối chứng	Chỉ số khảo sát không tương đương tỷ lệ chặn test
Pixy [3]	Phân tích tĩnh bảo mật web	Vai trò SAST và dữ liệu không tin cậy	Rule lab không phải phân tích luồng dữ liệu đầy đủ
Saner [4]	Phối hợp tĩnh–động để kiểm chứng sanitization	Đối chiếu lọc tên với hành vi HTTP	Không hiện thực thuật toán Saner
Dahse và Holz [5]	Dữ liệu lưu trước, dùng sau trong thao tác nhạy cảm	Upload kết hợp include trong lab	Không phân tích kho dữ liệu tổng quát

Từ khảo sát, đồ án lựa chọn một phạm vi có thể hoàn thành trong thời lượng môn học: xây dựng các kịch bản nhỏ nhưng quan sát được, đối chiếu trên hai nền tảng và đánh giá phòng vệ theo từng lớp. Phạm vi này ưu tiên khả năng giải thích nguyên nhân hơn quy mô phát hiện tự động. Bằng chứng cần gắn với phiên bản mã, cấu hình máy chủ và điều kiện kiểm thử để người khác có thể kiểm tra lại.

Khoảng trống mà sản phẩm học phần hướng tới là nhu cầu minh họa thống nhất giữa upload, lưu trữ, xử lý lại và phòng vệ. Một cảnh báo SAST được nối với vị trí mã cụ thể; một trường hợp tệp được chấp nhận được nối với tác động thực tế; một cơ chế phòng vệ được nối với test xác minh. Đây là đóng góp thực hành và giáo dục, không phải tuyên bố lấp đầy khoảng trống nghiên cứu của toàn bộ lĩnh vực.

Việc so sánh các công trình cũng cho thấy cần tránh một thước đo duy nhất. Tỷ lệ test đạt phản ánh mức phù hợp với kỳ vọng đã thiết kế; số cảnh báo phản ánh khả năng khớp rule; số lỗi phát hiện trong ứng dụng thực tế còn phụ thuộc tập dữ liệu và cách xác nhận. Vì vậy, báo cáo không đặt các con số khác bản chất cạnh nhau để suy ra công cụ nào tốt hơn.

CHƯƠNG 4. THỰC NGHIỆM

4.1. Thiết kế nghiên cứu và môi trường

Thiết kế thực nghiệm sử dụng endpoint cố ý có lỗ hổng làm đối chứng với endpoint phòng vệ. Người gửi có thể thay đổi tên tệp, Content-Type, nội dung và kích thước yêu cầu. Tài sản cần bảo vệ gồm vị trí lưu hợp lệ, tính toàn vẹn ứng dụng, tài nguyên xử lý và cách phục vụ nội dung. Mục tiêu là xác minh từng thuộc tính trong môi trường đã xác định, không mô phỏng toàn bộ một cuộc xâm nhập.

Ứng dụng phòng vệ giả định nghiệp vụ chỉ cần nhận ảnh JPEG hoặc PNG. Bộ kiểm thử dùng marker vô hại để phân biệt tệp được lưu, được phục vụ và được diễn giải. Với PHP, marker được in ra cho phép quan sát xử lý phía máy chủ mà không chạy lệnh hệ điều hành. Với Node.js, HTML marker chỉ xác minh lưu và trả lại nội dung, không chứng minh JavaScript đã thực thi.

Bảng 4.1. Môi trường thực nghiệm

Thành phần	Cấu hình khai báo	Vai trò
Máy điều phối	Windows, PowerShell, Python 3.11+	Khởi chạy và thu thập kết quả
PHP lab	php:8.2-apache; GD, Fileinfo	Upload và xử lý ảnh
Node lab	node:20-alpine; Express, Multer, Sharp	Upload và tải xuống
Compose	127.0.0.1:18081 → 80; 18082 → 3000	Hai dịch vụ độc lập, chỉ loopback
Runner	Python, thư viện chuẩn	HTTP, preflight và báo cáo
Semgrep	Hai rule PHP/JavaScript	Phân tích mã nguồn

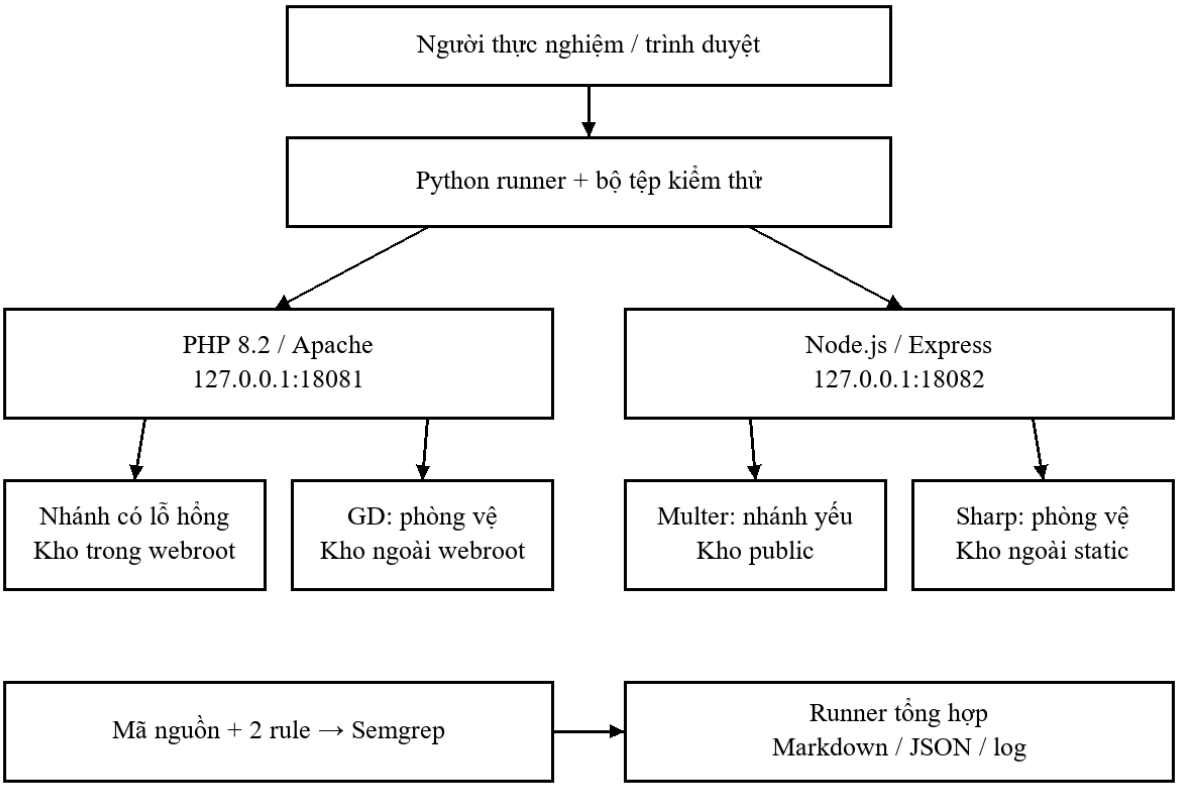
Hai ứng dụng không trao đổi nghiệp vụ và không sử dụng cơ sở dữ liệu. Mỗi container có vùng lưu riêng; Compose không khai báo volume lưu trữ bền vững. Tag image và dependency chưa được cố định hoàn toàn, vì vậy phiên bản khai báo không thay thế bản ghi phiên bản thực tế của lần chạy. Việc xóa container có thể làm mất dữ liệu upload.

4.2. Mô hình đe dọa

Client được xem là không tin cậy. Hệ thống không giả định filename, MIME hoặc byte đầu tệp phản ánh đúng nội dung. Lab chỉ kiểm tra các đường đi đã thiết kế; không đánh giá đầy đủ người dùng đã xác thực, phân quyền nhiều vai trò hoặc hạ tầng

reverse proxy. Hai cổng công bố trên loopback giới hạn truy cập từ máy cục bộ, nhưng không biến endpoint có lỗ hổng thành thành phần phù hợp cho production.

4.3. Kiến trúc hệ thống



Hình 4.1. Kiến trúc logic của B22 UFU Lab.

Khối điều phối gồm người thực nghiệm, trình duyệt và Python runner. Runner gửi bộ tệp kiểm thử tới hai dịch vụ, kiểm tra phản hồi và kết hợp bằng chứng lưu trữ. Khối phân tích tĩnh nhận mã nguồn và hai rule Semgrep; kết quả quét được runner tổng hợp với kết quả HTTP. Semgrep không nằm trên đường xử lý request và không tự chặn upload.

Trong PHP lab, nhánh yếu lưu tệp dưới webroot; nhánh phòng vệ sử dụng Fileinfo và GD rồi lưu ra ngoài webroot. LFI simulator minh họa việc xử lý lại tệp trong thư mục uploads. Trong Node lab, Multer của nhánh yếu lưu vào public; nhánh phòng vệ sử dụng Sharp rồi ghi ngoài vùng static. Hai dịch vụ là đối tượng so sánh, không phải frontend và backend của nhau.

Node có endpoint tải xuống theo ID và header kiểm soát; PHP hiện chưa có endpoint tương ứng. Sơ đồ không bổ sung cơ sở dữ liệu, WAF, antivirus hoặc xác thực

vì các thành phần đó chưa được triển khai. Các đường nối biểu diễn chức năng và không khẳng định có kho tệp dùng chung.

4.4. Các kịch bản có lỗ hổng

PHP có bảy kịch bản: tin Content-Type, double extension, xóa chuỗi yếu, mô phỏng null byte, magic bytes, ảnh có nội dung bổ sung và path traversal. Mỗi kịch bản cô lập một quyết định kiểm tra để có thể giải thích nguyên nhân. Tệp được chấp nhận được chuyển vào vùng upload dưới webroot. Apache gán handler PHP cho .php và vô hiệu hóa PHP ở uploads/sub, tạo hai vùng có chính sách thực thi khác nhau.

Trong traversal của PHP, đường dẫn người gửi được ghép sau sub. Thành phần thư mục cha trong mẫu kiểm thử có thể đưa tệp lên uploads, thay đổi ngữ cảnh handler. Kết quả marker tại vị trí mới minh họa sự kết hợp giữa lỗi đường dẫn và cấu hình thực thi. Đây là hành vi trong cấu hình lab, không phải khẳng định mặc định của mọi máy chủ Apache.

Node có public-upload lưu vào public/uploads và traversal-upload dùng originalname với preservePath. Việc giữ đường dẫn là cấu hình cố ý không an toàn để phục vụ minh họa; không nên mô tả đây là hành vi mặc định của Multer. Endpoint traversal chỉ dựa vào MIME client, trong khi vị trí lưu chịu ảnh hưởng của tên gửi lên. Kết luận traversal cần kiểm tra đường dẫn thực tế.

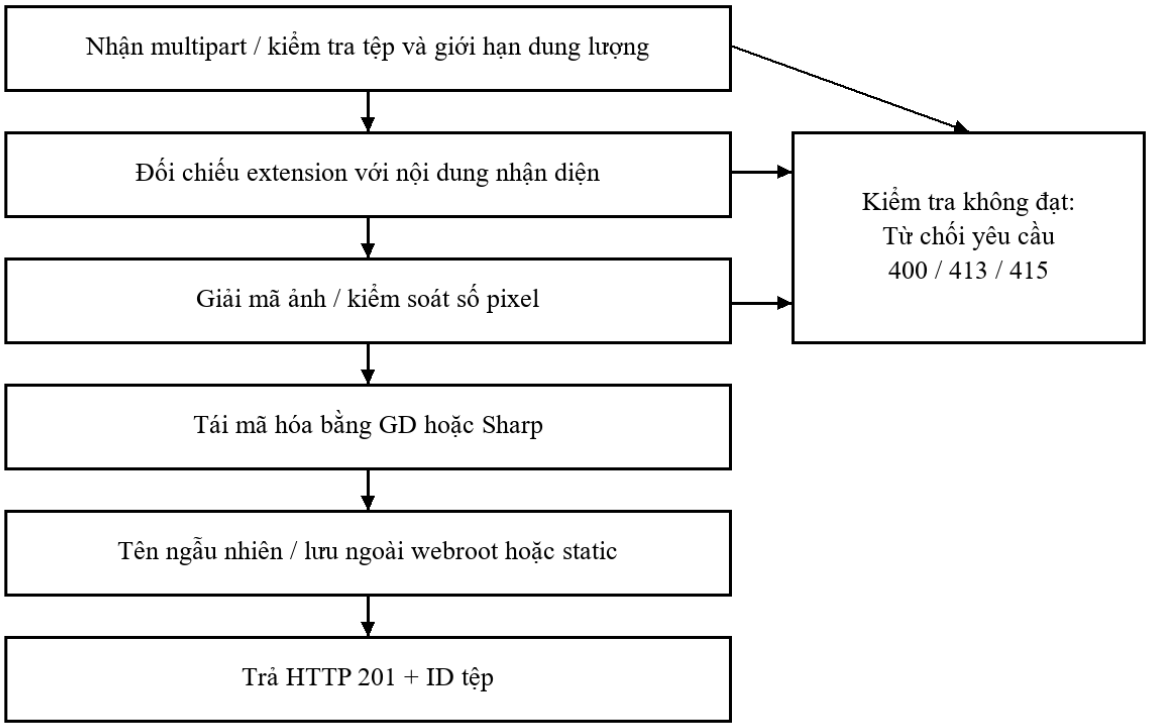
Bảng 4.2. Nhóm kiểm thử và bằng chứng

Nhóm kịch bản	Dữ liệu thay đổi	Bằng chứng cần thu
Content-Type và extension	Tên hoặc MIME khác nội dung	Mã HTTP; stored; truy xuất tệp
Magic bytes, ảnh có dữ liệu nối	Cấu trúc và nội dung tệp	Giải mã ảnh; marker trước/sau
Traversal	Thành phần đường dẫn trong tên	Đường dẫn lưu và vùng thực thi
Upload kết hợp LFI	Tệp được xử lý sau khi lưu	Marker và ranh giới canonical path
Giới hạn và đầu vào lỗi	Dung lượng; field; ảnh rỗng/hỏng	400, 413 hoặc 415 theo điều kiện
SAST	Hai mẫu cấu trúc mã nguồn	Rule ID, vị trí, nội dung cảnh báo

Tên kịch bản exif_polyglot không chứng minh rằng dữ liệu nằm trong EXIF. Phân tích cần căn cứ mẫu thực tế: lab có ảnh với dữ liệu PHP nối thêm. Tương tự,

null_byte_sim chỉ mô phỏng sai lệch giữa kiểm tra và sử dụng tên tệp, không phải bypass của PHP hiện đại.

4.5. Luồng phòng vệ và triển khai



Hình 4.2. Luồng xử lý logic của upload phòng vệ.

PHP xác minh trạng thái upload, ngưỡng byte và extension, sau đó đối chiếu MIME bằng Fileinfo, thông tin ảnh và khả năng giải mã GD. Ảnh bị giới hạn số pixel và chiều; ảnh được tái mã hóa, đặt tên ngẫu nhiên và lưu ngoài webroot. Node sử dụng Multer memoryStorage có giới hạn, đối chiếu extension với chữ ký JPEG/PNG rồi dùng Sharp giải mã và chuyển thành JPEG. Các lỗi lưu trữ không được coi là lỗi định dạng ảnh.

Bảng 4.3. Đối chiếu nhánh phòng vệ

Thuộc tính	**PHP**	**Node.js**
Giới hạn tệp ứng dụng	1 MiB	2 MiB
Giới hạn ảnh	4 triệu pixel; mỗi chiều ≤ 4096	4 triệu pixel đầu vào
Định dạng đầu ra	JPEG hoặc PNG tương ứng	JPEG
Tên lưu	16 byte ngẫu nhiên dạng hex	UUID v4
Quyền tệp	0600	0600; ghi độc quyền wx
Tải xuống	Chưa có endpoint tương ứng	ID; attachment; nosniff; CSP

Các mã 400, 413 và 415 trong hình thể hiện nhóm lỗi dữ liệu đầu vào. Chi tiết mã trả về phụ thuộc nền tảng và bước phát hiện lỗi. Tên ngẫu nhiên, header và lưu ngoài vùng public không thay thế xác thực hay phân quyền. Ngưỡng dung lượng khác nhau nên không dùng tỷ lệ chấp nhận để so sánh công bằng nếu chưa chuẩn hóa đầu vào.

4.6. Quy trình tái lập và tiêu chí đánh giá

Trước khi chạy, cần Docker Desktop ở chế độ Linux containers, Docker Compose, Python và kết nối Internet để tải image/dependency lần đầu. Từ PowerShell, sử dụng đường dẫn tuyệt đối đến Compose và runner như bên dưới. Preflight kiểm tra Docker và hai dịch vụ; nếu chưa sẵn sàng, runner trả mã 2 và giữ nguyên báo cáo cũ.

```
docker compose -f 'D:\UIT\NT213\doan\b22-ufu-lab\docker-compose.yml' up -d --build
python 'D:\UIT\NT213\doan\b22-ufu-lab\tests\run_experiments.py'
```

Khi thực nghiệm chạy được, runner sinh payload, gửi yêu cầu, kiểm tra phản hồi và một số thuộc tính tệp trong container, chạy Semgrep rồi ghi Markdown, JSON và log trong thư mục docs. Mã thoát 0 biểu thị mọi kiểm tra đạt, 1 biểu thị có kiểm tra thất bại và 2 biểu thị hạ tầng chưa sẵn sàng. Không đọc một báo cáo cũ như bằng chứng của lần chạy bị chặn.

PASS nghĩa là hành vi quan sát khớp kỳ vọng của test. Với endpoint cố ý có lỗ hổng, việc diễn giải marker có thể là PASS của bộ kiểm thử nhưng đồng thời là bằng chứng không an toàn. FAIL là sai khác với kỳ vọng; BLOCKED là chưa thể kiểm tra vì điều kiện tiền đề. Mã HTTP 200 hoặc 201 riêng lẻ không chứng minh khai thác thành công: cần phân biệt tiếp nhận, lưu, truy xuất và diễn giải nội dung.

4.7. Kịch bản trình diễn

Buổi demo bắt đầu bằng xác nhận hai dịch vụ chỉ công bố trên loopback. Tiếp theo, trình bày một mẫu PHP có MIME giả để quan sát marker, đối chiếu với nhánh phòng vệ từ chối tệp giả ảnh. Sau đó, dùng ảnh chứa dữ liệu bổ sung để quan sát tái mã hóa và vị trí lưu tách biệt. Cuối cùng, trình bày header tải xuống của Node và hai cảnh báo Semgrep tương ứng vị trí mã cố ý yếu.

Bằng chứng demo cần gồm ảnh hoặc bản ghi phản hồi HTTP, vị trí tệp, kết quả runner và phiên bản môi trường. Báo cáo này không chèn ảnh chụp giao diện chưa

được tạo thực tế. Sau demo cần dừng lab; không chuyển cấu hình bind sang mọi interface và không dùng payload có chức năng điều khiển hệ thống.

CHƯƠNG 5. KẾT QUẢ VÀ BÀN LUẬN

5.1. Nguồn bằng chứng và trạng thái xác minh

Báo cáo B22_EXPERIMENTAL_RESULTS.md và verification.json trong thư mục docs lưu kết quả 25 kiểm tra PASS của lần thực nghiệm trước. README ngày 02/11/2026 ghi nhận đã bổ sung 11 kiểm tra tích hợp biên nhưng chưa chạy lại HTTP/Semgrep sau thay đổi. Do đó, 25 PASS được trình bày là bằng chứng lịch sử của lab, không xác nhận đầy đủ phiên bản hiện tại. Ngày biên soạn 03/11/2026 đã chạy lại sáu unit test của preflight và tất cả đạt; đây không phải chạy lại HTTP hoặc PHP trong container.

Bảng 5.1. Phân loại trạng thái bằng chứng

Hạng mục	Bằng chứng	Diễn giải
Tích hợp HTTP và SAST đã lưu	25 PASS trong báo cáo cũ	Có bằng chứng cho lần chạy trước
Kiểm tra tích hợp biên bổ sung	11 trường hợp theo README	Chưa ghi nhận chạy lại đầy đủ
Unit test preflight	6/6 đạt khi biên soạn	Xác minh điều phối bằng mock
Đánh giá production	Chưa có	Không kết luận sẵn sàng triển khai

5.2. Kết quả của nhánh PHP

Bằng chứng đã lưu cho thấy nhánh tin Content-Type chấp nhận PHP khi phía gửi khai báo MIME ảnh; truy xuất tệp trả marker đúng kỳ vọng. Double extension cũng tạo marker trong cấu hình AddHandler của lab. Hai kết quả thể hiện rằng việc chấp nhận dựa trên thuộc tính tên hoặc MIME có thể không phù hợp với cách máy chủ thực sự diễn giải tệp.

Kịch bản traversal đưa tệp ra khỏi thư mục con tắt PHP tới vùng uploads, nơi marker được diễn giải. LFI simulator chấp nhận tệp nằm trong uploads và từ chối đường dẫn ngoài ranh giới. Khi ảnh có PHP nối thêm được đưa vào include, kết quả đã lưu ghi nhận marker. Điều này cho thấy ranh giới đường dẫn chặt chẽ không tự bảo đảm mọi nội dung bên trong thư mục đều an toàn để thực thi.

Nhánh phòng vệ PHP nhận ảnh hợp lệ, tái mã hóa, kiểm tra marker không còn và xác minh tồn tại tệp ngoài webroot. Tệp PHP đổi tên PNG và tệp PHP thông thường bị từ chối với 415 trong lần chạy đã lưu. Kết luận chỉ áp dụng cho mẫu đã dùng; không suy rộng sang mọi định dạng hay mọi cách che giấu dữ liệu.

5.3. Tổng hợp kết quả đã lưu

Bảng 5.2. Phân nhóm 25 kiểm tra trong báo cáo có sẵn

Nhóm kiểm tra	Số test	Kết quả đã lưu
PHP: MIME giả và truy xuất marker	2	PASS
PHP: chấp nhận đầu vào của 7 kịch bản	7	PASS
PHP: double extension, traversal, LFI, tắt PHP ở sub	6	PASS
PHP: phòng vệ, tái mã hóa, từ chối PHP/giả ảnh	3	PASS
Node: public upload, tái mã hóa, header, giả MIME, traversal, quá cỡ	6	PASS
SAST: phát hiện cả hai mẫu cố ý yếu	1	PASS
Tổng	25	Bằng chứng của lần chạy trước

Các dòng tổng hợp được phân nhóm từ bảng kết quả nguồn, không phải các lần chạy mới. Một test có thể bao gồm nhiều điều kiện, ví dụ HTTP 201, stored đúng và tệp tồn tại ngoài webroot. Số 25 vì vậy không phải số lỗ hổng độc lập, cũng không phải mẫu thống kê đại diện cho toàn bộ lớp tấn công UFU.

5.4. Kết quả Node.js và Semgrep

Endpoint public-upload lưu và phục vụ lại HTML marker. Nội dung này là chú thích HTML, không phải bằng chứng XSS đã thực thi. Nhánh Sharp tái mã hóa PNG thành JPEG, xác minh marker không còn và lưu ngoài vùng static. Endpoint tải xuống có Content-Disposition dạng attachment, X-Content-Type-Options: nosniff và CSP hạn chế. Các test giả MIME/extension và quá dung lượng ghi nhận bị từ chối trong báo cáo đã lưu.

Kiểm tra traversal của Node cần được đọc cùng điều kiện preservePath và đường dẫn ghi thực tế. Không nên chuyển kết quả PASS thành tuyên bố rằng mọi cấu hình

Multer đều có traversal. Tương tự, hai cảnh báo Semgrep chỉ xác nhận hai rule khớp mẫu cố ý yếu. Log cũ ghi phiên bản Semgrep 1.181.1 và hai vị trí cảnh báo; số dòng có thể thay đổi sau chỉnh sửa mã.

Một rule nhận diện move_uploaded_file khi đích được xây từ thư mục uploads; rule còn lại nhận diện cấu hình Multer có dest trong public/uploads. Chúng hữu ích cho minh họa và kiểm tra hồi quy cấu trúc, nhưng không phân tích đầy đủ việc kiểm tra nội dung trước lưu, cấu hình handler hoặc mọi biến thể API. Không đủ dữ liệu để báo cáo precision, recall hay tỷ lệ phát hiện tổng quát.

5.5. Trả lời câu hỏi nghiên cứu và hạn chế

Đối với RQ1, các trường hợp MIME giả, double extension và kiểm tra sơ bộ nội dung cho thấy kiểm tra đơn lẻ không bảo đảm an toàn trong lab. Với RQ2, cấu hình Apache và ngữ cảnh include làm thay đổi tác động sau lưu; Express phục vụ tệp không đồng nghĩa thực thi PHP. Các quan sát này hỗ trợ cách đánh giá theo cả chuỗi tiếp nhận–lưu–xử lý thay vì chỉ xem request upload.

Đối với RQ3, bằng chứng cũ hỗ trợ hiệu quả của nhánh phòng vệ trên mẫu đã kiểm thử: tệp giả bị từ chối, ảnh được tái mã hóa và lưu tách biệt. Tuy nhiên, cần chạy lại các kiểm tra tích hợp sau thay đổi trước khi xác nhận phiên bản hiện tại. Với RQ4, Semgrep tìm đúng hai mẫu đã thiết kế, nhưng chưa có tập ứng dụng được gán nhãn độc lập để đánh giá độ bao phủ hoặc sai số.

5.6. Các nguy cơ ảnh hưởng tính đúng đắn

Về tính đúng đắn nội tại, một kết quả thành công có thể phụ thuộc cấu hình cố ý yếu hoặc test chỉ kiểm tra thuộc tính hẹp. Bằng chứng HTTP cần đi kèm nội dung phản hồi và vị trí lưu. Việc không thấy marker trong đầu ra chỉ xác nhận sự vắng mặt của chuỗi đó, không chứng minh không còn dữ liệu độc hại khác. Unit test sử dụng mock xác minh nhánh điều phối, không thay thế thử nghiệm Docker thật.

Về khả năng khái quát, bộ mẫu nhỏ và hai nền tảng không đại diện cho mọi hệ thống upload. Các ngưỡng dung lượng khác nhau khiến so sánh tỷ lệ chấp nhận cần thận trọng. Chưa đo thời gian xử lý, tải đồng thời, mức dùng bộ nhớ hoặc ảnh hưởng của ảnh nén có kích thước giải nén lớn. Chưa có đánh giá người dùng hoặc usability của chính sách từ chối.

Về tái lập, image và dependency chưa khóa hoàn toàn, chưa có npm lockfile theo README. Bằng chứng cũ và mã mới không nên gộp thành một kết quả duy nhất. Mỗi lần chạy chính thức cần ghi commit hoặc hash mã nguồn, phiên bản công cụ, cấu hình và thời gian. Cơ chế preflight bảo toàn báo cáo tránh mất bằng chứng nhưng cũng đòi hỏi người đọc nhận biết báo cáo có thể thuộc lần chạy trước.

5.7. Ý nghĩa thực tiễn

Một thiết kế upload an toàn cần đồng thời hạn chế nội dung, tài nguyên, vị trí lưu và cách truy xuất. Đồ án minh họa được các lớp này nhưng chưa có xác thực, phân quyền, quota, rate limit hoặc quét malware. Khi áp dụng vào sản phẩm thực, cần bổ sung các cơ chế theo nghiệp vụ, vô hiệu hóa thực thi tại vùng lưu, cập nhật thư viện và kiểm thử hồi quy thay vì sao chép trực tiếp endpoint lab.

CHƯƠNG 6. KẾT LUẬN

6.1. Tổng kết

Đề tài xây dựng và phân tích môi trường thực nghiệm tải tệp không an toàn trên PHP/Apache và Node.js/Express. Hai nhánh đối chứng cho phép quan sát mối liên hệ giữa chính sách kiểm tra, tên tệp, vị trí lưu và cách xử lý sau upload. Bộ kiểm thử tự động cùng Semgrep hỗ trợ liên kết bằng chứng hành vi với mẫu mã nguồn, phù hợp mục tiêu học phần trong thời gian ngắn.

Kết quả đã lưu gồm 25 kiểm tra đạt, thể hiện các hành vi mong đợi của cả nhánh có lỗ hổng và nhánh phòng vệ trong lần chạy trước. Sáu unit test preflight đã được chạy lại khi biên soạn và đạt. Do có các thay đổi và kiểm tra tích hợp bổ sung chưa được xác minh lại, báo cáo không tuyên bố mọi chức năng của phiên bản hiện tại đã được kiểm thử end-to-end.

Kết luận chính là không thể suy ra an toàn upload từ một thuộc tính như extension hoặc Content-Type. Phòng vệ cần kết hợp kiểm tra nội dung, giải mã và tái mã hóa, giới hạn tài nguyên, tên lưu do máy chủ kiểm soát và tách biệt vùng lưu. Tác động phụ thuộc ngữ cảnh sử dụng: tệp được phục vụ, được include hoặc được xử lý bởi thư viện ảnh có các rủi ro khác nhau.

6.2. Hướng phát triển

Ưu tiên trước mắt là chạy lại đầy đủ bộ tích hợp trên bản mã cuối, xử lý FAIL/BLOCKED và lưu bằng chứng phiên bản. Tiếp theo, mở rộng bộ mẫu hợp lệ và không hợp lệ để giảm nguy cơ bộ lọc quá chặt hoặc bỏ sót. Các phép đo hiệu năng cần kiểm soát cấu hình và kích thước đầu vào để so sánh công bằng hai nền tảng.

Về sản phẩm, cần bổ sung xác thực và phân quyền tải tệp, quota, rate limit, chính sách lưu giữ và ghi log. Về nghiên cứu, có thể xây dựng tập mã gán nhãn để đánh giá precision/recall của rule, mở rộng phân tích luồng dữ liệu và khảo sát tác động của cấu hình web server. Việc cố định dependency và quản lý phiên bản là điều kiện quan trọng để kết quả có thể được kiểm chứng lâu dài.

TÀI LIỆU THAM KHẢO

[1] J. Huang, Y. Li, J. Zhang và R. Dai, "UChecker: Automatically Detecting PHP-Based Unrestricted File Upload Vulnerabilities," IEEE/IFIP DSN, 2019. DOI: 10.1109/DSN.2019.00064.

[2] I. Riadi và E. I. Aristianto, "An Analysis of Vulnerability Web Against Attack Unrestricted Image File Upload," Computer Engineering and Applications Journal, tập 5, số 1, 2016. DOI: 10.18495/comengapp.v5i1.161.

[3] N. Jovanovic, C. Kruegel và E. Kirda, "Pixy: A Static Analysis Tool for Detecting Web Application Vulnerabilities," IEEE Symposium on Security and Privacy, 2006. DOI: 10.1109/SP.2006.29.

[4] D. Balzarotti, M. Cova, V. Felmetsger, N. Jovanovic, E. Kirda, C. Kruegel và G. Vigna, "Saner: Composing Static and Dynamic Analysis to Validate Sanitization in Web Applications," IEEE Symposium on Security and Privacy, 2008. DOI: 10.1109/SP.2008.22.

[5] J. Dahse và T. Holz, "Static Detection of Second-Order Vulnerabilities in Web Applications," 23rd USENIX Security Symposium, 2014, tr. 989–1003. https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/dahse

[6] MITRE, "CWE-434: Unrestricted Upload of File with Dangerous Type." https://cwe.mitre.org/data/definitions/434.html

[7] OWASP Foundation, "File Upload Cheat Sheet," OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

[8] B22 UFU Lab, mã nguồn, README và báo cáo thực nghiệm nội bộ. Thư mục dự án: D:\UIT\NT213\doan\b22-ufu-lab. Đối chiếu khi biên soạn ngày 03/11/2026.

Ghi chú nguồn: [1]–[5] là năm công trình học thuật. [6]–[7] là tài liệu kỹ thuật, không tính thay thế paper. Thông tin thư mục [1]–[4] được đối chiếu qua Crossref/DOI; nội dung tóm tắt [2], trang hội nghị [5] và hướng dẫn [7] được sử dụng trong khảo sát. Báo cáo không đưa số liệu hay chi tiết thuật toán chưa được xác minh từ toàn văn.